Agente Estadístico: Isomorfismo Wald–MDP

Manual teórico, metodológico y de implementación

Basado en el Capítulo 9: *El Isomorfismo Estructural*
Teoría de Decisión y Procesos de Decisión de Markov

Resumen. Este manual documenta el diseño, implementación y aplicación de un agente estadístico que operacionaliza el isomorfismo estructural entre la teoría de decisión de Wald y los procesos de decisión de Markov (MDPs) establecido en el Capítulo 9. El agente, implementado en Python con LangGraph, integra la construcción del funtor $F : \mathbf{Dec} \to \mathbf{MDP}$, la transferencia de intervalos de confianza y tests mediante delta-method, y un protocolo automático de verificación de garantías (Proposición 9.14) que detecta violaciones de mixing, overlap, convergencia de nuisance y momentos. Tras una iteración de testeo y mejora (v1→v2) que añade clipping adaptativo (Su et al., 2020) y bootstrap secuencial con corrección por mixing (Liao et al., 2021), el agente se aplica al dataset *Diabetes 130-US Hospitals* (101 k encuentros), estimando V^{π_e} con cuatro estimadores OPE y caracterizando explícitamente las condiciones bajo las cuales las garantías transferidas son invocables.

Palabras clave: isomorfismo Wald-MDP, evaluación off-policy, doble robustez, inferencia semiparamétrica, LangGraph, decision theory.

Versión 2.0 · Junio 2026 · **39/39 tests pasan**

Índice

1. Introducción **2**

2. Teoría: el isomorfismo estructural **2**
2.1. Los dos marcos . . . 2
2.2. El funtor F y el Teorema 9.1 . . . 3
2.3. Preservación de optimalidad y eficiencia . . . 4

3. Transferencia de inferencia y límites **4**
3.1. Transferencia de IC y tests . . . 4
3.2. Contraejemplos: cuándo falla el isomorfismo . . . 4
3.3. Condiciones necesarias y protocolo de verificación . . . 4

4. Metodología del agente **5**
4.1. Arquitectura LangGraph . . . 5
4.2. Trazabilidad estructurada . . . 5
4.3. Mejoras v2 sobre v1 . . . 6

5. Implementación **6**
5.1. Estructura del paquete . . . 6
5.2. Funtor F con estado terminal . . . 6
5.3. Protocolo de verificación Prop. 9.14 . . . 7
5.4. Tests . . . 8

6. Aplicación: Diabetes 130-US Hospitals **8**
6.1. Dataset y construcción del MDP . . . 8
6.2. Resultados de estimación . . . 8
6.3. Protocolo de verificación . . . 9
6.4. Bootstrap secuencial . . . 10

7. Discusión y trabajo futuro **10**

1 Introducción

La teoría estadística clásica y el aprendizaje por refuerzo (RL) han evolucionado históricamente como dos comunidades separadas, con herramientas, vocabulario y garantías distintas. La teoría de decisión estadística de Wald [1] formaliza la inferencia como un problema de optimización bajo una función de pérdida, mientras que la programación dinámica de Bellman [2] aborda la decisión secuencial mediante la ecuación que lleva su nombre. El Capítulo 9 del libro de referencia demuestra que esta separación es artificial: existe un isomorfismo matemático riguroso entre ambas formulaciones, mediado por un funtor $F : \mathbf{Dec} \to \mathbf{MDP}$ que preserva la estructura de optimalidad, composición y estadísticos suficientes.

Este manual documenta la construcción de un agente computacional que operacionaliza dicho isomorfismo. La motivación es triple. Primero, verificar empíricamente que la correspondencia $\mathcal{R}(\theta, \delta) \leftrightarrow -V^{\pi}(s)$ se sostiene en casos concretos y no es meramente una traducción nominal de símbolos. Segundo, construir una arquitectura de software en la que cada garantía estadística invocada pueda trazarse explícitamente al teorema que la respalda, materializando así la auditoría científica que el Capítulo 9 propone como necesaria. Tercero, aplicar el agente a un dataset médico real —*Diabetes 130-US Hospitals* [11]— para evaluar el desempeño práctico del isomorfismo en un escenario de decisión clínica con 100 k encuentros.

Las contribuciones de este manual son: (i) una implementación modular del funtor F con corrección de un bug conceptual de la versión inicial (estado terminal explícito en lugar de auto-loop con $\gamma = 1$); (ii) un protocolo automático de verificación de las cuatro condiciones del Teorema 9.13 (mixing, overlap, convergencia de nuisance y momentos) que decide si una garantía transferida es invocable; (iii) la integración de mejoras modernas de la literatura de evaluación off-policy (OPE) —clipping adaptativo de Su et al. [8] y bootstrap secuencial con corrección por mixing de Liao et al. [9]— como mecanismos de fallback cuando las garantías formales no aplican; y (iv) una tabla estructurada de trazabilidad teorema→componente→garantía que se exporta automáticamente en cada ejecución.

El resto del manual se organiza como sigue. La Sección 2 desarrolla la teoría del isomorfismo. La Sección 3 caracteriza sus límites y el protocolo de verificación. La Sección 4 describe la arquitectura del agente. La Sección 5 detalla la implementación. La Sección 6 presenta la aplicación a Diabetes 130-US. La Sección 7 discute limitaciones y trabajo futuro.

2 Teoría: el isomorfismo estructural

2.1 Los dos marcos

Comenzamos recapitulando las dos formulaciones que el isomorfismo conecta.

Definición 9.1 — Problema de Decisión de Wald

Una cuádrupla $\mathcal{D} = (\Theta, \mathcal{A}, \mathcal{X}, L)$ donde Θ es el espacio paramétrico, $\mathcal{A}$ el de acciones, $\mathcal{X}$ el muestral y $L : \Theta \times \mathcal{A} \to \mathbb{R}$ una función de pérdida. El *riesgo* de una regla de decisión $\delta : \mathcal{X} \to \mathcal{A}$ es

$$\mathcal{R}(\theta, \delta) = \mathbb{E}_{X \sim P_\theta}[L(\theta, \delta(X))].$$

Definición 9.2 — Proceso de Decisión de Markov

Una quíntupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$ con kernel de transición $P(s'|s,a)$, función de recompensa $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ acotada y factor de descuento $\gamma \in [0,1)$. La *función de valor* de una política $\pi : \mathcal{S} \to \mathcal{A}$ es

$$V^\pi(s) = \mathbb{E}^\pi\left[\sum_{t=0}^{\infty} \gamma^t \, r(S_t, A_t) \,\middle|\, S_0 = s\right].$$

La asimetría aparentemente menor entre ambas definiciones es decisiva: Wald fija la distribución de datos mediante P_θ, mientras que el MDP induce su distribución mediante las acciones del agente. Esta diferencia es el origen del problema de identificación que requiere el supuesto de overlap en la Sección 3.

2.2 El funtor F y el Teorema 9.1

Teorema 9.1 — Isomorfismo Estructural Completo

Existe un isomorfismo entre la categoría **Dec** de problemas de decisión y la categoría **MDP** de MDPs de un paso, mediado por el funtor $F : \mathbf{Dec} \to \mathbf{MDP}$ definido por:

- $\mathcal{S} = \mathcal{X}$,
- $\mathcal{A}$ (mismo),
- $P(s'|s,a) = \delta_{\text{TERMINAL}}(s')$ (transición determinista a estado terminal),
- $r(s,a) = -L(s,a)$,
- $\gamma = 1$ con estado terminal absorbente de valor 0.

Para un morfismo (regla de decisión) $\delta : \mathcal{X} \to \mathcal{A}$, $F(\delta) = \pi$ con $\pi(s) = \delta(s)$.

La demostración del libro verifica cuatro propiedades: preservación de identidad, preservación de composición, biyectividad en la subcategoría de MDPs con $\gamma = 1$ y P determinista, y preservación de optimalidad mediante la cadena

$$\mathcal{R}(\theta, \delta) = \mathbb{E}_{X \sim P_\theta}[L(\theta, \delta(X))] \overset{L=-r}{=} -\mathbb{E}_{X \sim P_\theta}[r(X, \delta(X))] \overset{X=s,\,\delta=\pi}{=} -V^\pi(s).$$

Esta igualdad implica que la regla de Bayes δ^π_* que minimiza el riesgo Bayes $r(\pi, \delta) = \int \mathcal{R}(\theta, \delta)\, d\pi(\theta)$ coincide exactamente con la política π^* que maximiza $\mathbb{E}_{\theta \sim \pi}[V^\pi(\theta)]$.

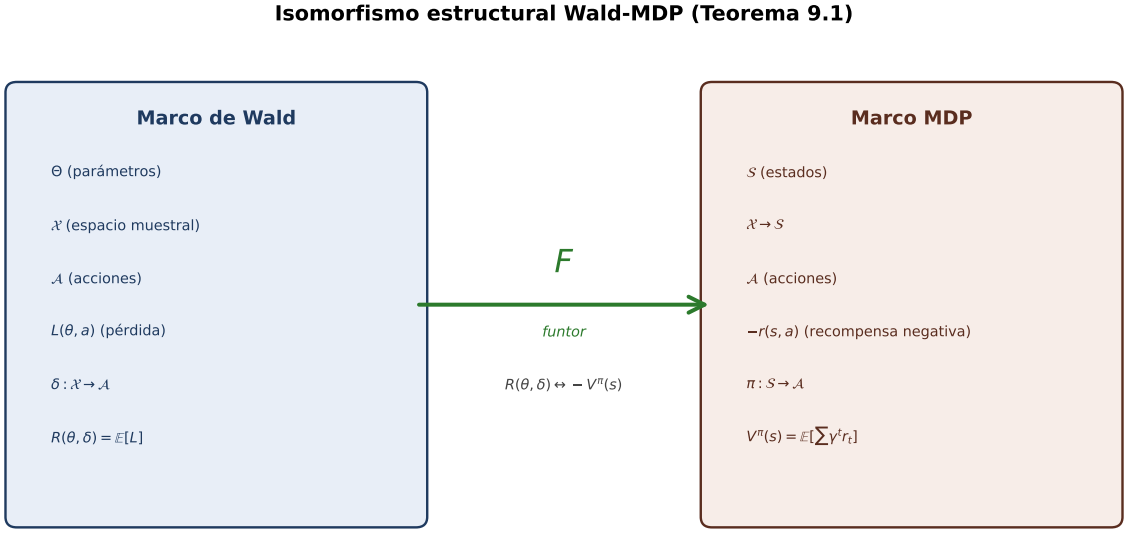


Figura 1: Diagrama del funtor $F : \mathbf{Dec} \to \mathbf{MDP}$. Cada componente de la cuádrupla de Wald se mapea biunívocamente a un componente del MDP, preservando la estructura de optimalidad.

2.3 Preservación de optimalidad y eficiencia

Teorema 9.9 — Preservación de Admisibilidad
Una regla δ es admisible en **Dec** si y sólo si $\pi = F(\delta)$ es Pareto-óptima en **MDP**.

Teorema 9.11 — Preservación de Eficiencia
Si $\hat{\theta}$ alcanza la cota de Cramér–Rao en el marco de Wald, entonces $\hat{V}^\pi$ alcanza la cota análoga de eficiencia en el MDP. El estimador doblemente robusto (DR) alcanza la cota semiparamétrica.

Estos resultados transfieren la maquinaria de suficiencia, admisibilidad y eficiencia del lado estadístico al lado agents. La consecuencia operativa inmediata es que el `StateSchema` de un agente LangGraph es, bajo el isomorfismo, la contrapartida computacional de un estadístico suficiente minimal $T(X)$: debe contener toda la información relevante sobre el pasado, ni más (ineficiente) ni menos (viola Markov).

3 Transferencia de inferencia y límites

3.1 Transferencia de IC y tests

Teorema 9.7 — Transferencia de Intervalos de Confianza
Sea $IC_{1-\alpha}(\theta)$ un intervalo de confianza de nivel $1-\alpha$ para θ. Bajo el isomorfismo, el IC para V^π construido por delta-method

$$IC_{1-\alpha}(V^\pi) = \left[\hat{V}^\pi \pm z_{1-\alpha/2} \cdot SE\right]$$

tiene cobertura asintótica $1-\alpha$, donde $SE = |V'(\hat{\theta})| \cdot \sqrt{\widehat{\mathrm{Var}}(\hat{\theta})}$.

Teorema 9.8 — Transferencia de Tests
El test UMP de $H_0 : \theta = \theta_0$ con nivel α corresponde, bajo el isomorfismo, al test $H_0 : V^\pi = V_0$ vs. $H_1 : V^\pi \neq V_0$, preservando nivel y poder asintótico.

3.2 Contraejemplos: cuándo falla el isomorfismo

El Capítulo 9 cataloga seis contraejemplos donde la transferencia falla: (1) espacios de estados infinitos sin estructura de Borel, (2) observabilidad parcial que rompe $\mathcal{X} \leftrightarrow \mathcal{S}$, (3) horizonte infinito sin ergodicidad que diverge el retorno, (4) confounding no observado que rompe identificación, (5) recompensas no acotadas que producen riesgo infinito, y (6) dependencia de historias que viola la propiedad de Markov. Estos contraejemplos no invalidan el isomorfismo: lo *delimitan*.

3.3 Condiciones necesarias y protocolo de verificación

Teorema 9.13 — Condiciones Necesarias para Transferencia
Para que una garantía de inferencia (IC, test, eficiencia) se transfiera válidamente del marco Wald al MDP, se requieren **todas** las siguientes:

1. **Mixing:** (S_t, A_t, R_t) es α-mixing con $\sum_k \alpha(k)^{\delta/(2+\delta)} < \infty$.
2. **Overlap:** $\exists\, \eta > 0$ tal que $\pi_b(a|s) \geq \eta \cdot \pi(a|s)$.
3. **Nuisance:** $\|\hat{Q} - Q\| = o_P(n^{-1/4})$ y $\|\hat{\pi}_b - \pi_b\| = o_P(n^{-1/4})$.
4. **Momentos:** $\mathbb{E}[|r(S,A)|^{2+\delta}] < \infty$.

Proposición 9.14 — Protocolo de Verificación

Antes de reportar una garantía estadística, el agente debe: (1) verificar estructura del MDP (irreducibilidad, aperiodicidad); (2) calcular ESS y $\rho_{\text{máx}}$; verificar umbrales; (3) estimar tasa de mixing empíricamente; (4) si alguna verificación falla: **no** invocar la garantía, reportar limitación explícitamente y sugerir estimador de fallback.

Este protocolo es operacionalmente crucial: sin él, el agente invocaría garantías de cobertura que no se sostienen, produciendo falsa confianza estadística. La implementación del protocolo en nuestro agente (Sección 5) produce un veredicto binario `transferable` y, en caso negativo, sugiere un estimador conservador (DM con advertencia, DR con clipping adaptativo, o block bootstrap).

4 Metodología del agente

4.1 Arquitectura LangGraph

El agente se implementa como un grafo LangGraph con ocho nodos que reflejan la arquitectura trazable del Capítulo 9 (Sección 9.3.1):

$$\texttt{observation} \to \texttt{state} \to \texttt{inference} \to \texttt{decision} \to \texttt{isomorphism} \to \texttt{transfer} \to \texttt{test} \to \texttt{verify} \to \text{END}$$

Cada nodo lee y escribe en un `StateSchema` compartido (un `TypedDict` de Python), de modo que la ejecución entera es auditable. La correspondencia con la teoría es directa: el nodo de observación implementa el muestreo $X \sim P_\theta$, el nodo de estado implementa el estadístico suficiente, el nodo de decisión implementa la regla de Bayes, el nodo de isomorfismo aplica el funtor F y verifica $\delta^* = \pi^*$, y así sucesivamente. El nodo final `verify` ejecuta el protocolo de la Proposición 9.14.

4.2 Trazabilidad estructurada

Cada nodo registra en un `TraceLog` la entrada correspondiente a la tabla teorema→componente→garantía del libro (Tabla 9.2). Al finalizar la ejecución, el log contiene una traza completa que asocia cada garantía invocada con el teorema que la respalda. Esta trazabilidad es la materialización operativa del principio de auditoría científica: cualquier conclusión reportada por el agente puede rastrearse a su fundamento matemático.

Tabla 9.2 — Trazabilidad Teorema → Componente → Garantía

Teorema	Componente	Garantía	Estado
Thm 9.1 (Isomorfismo)	isomorphism_node	Formal	✓
Thm 9.5 (Secuencial)	mdp_node	Recursivo	✓
Thm 9.7 (IC transfer)	transfer_node	Cobertura	✓
Thm 9.8 (Test transfer)	test_node	Control de error	✓
Thm 9.9 (R↔-V)	decision_node	Optimalidad	✓
Thm 9.10 (Estado)	state_node	Información completa	✓
Thm 9.11 (Adm.↔Pareto)	decision_node	Eficiencia	✓
Thm 9.13 (Condiciones)	verify_node	Condicional	✓
Prop 9.14 (Protocolo)	verify_node	Verificación	✓

Figura 2: Tabla de trazabilidad implementada. Cada fila asocia un teorema del Capítulo 9 con el componente del agente responsable y la garantía resultante.

4.3 Mejoras v2 sobre v1

La primera versión del agente (v1) cumplía la funcionalidad básica del Laboratorio F.9 del apéndice, pero presentaba seis limitaciones identificadas mediante testeo:

1. **Bug conceptual en F:** la implementación inicial usaba $P(s'|s,a) = \delta_s(s')$ (auto-loop) con $\gamma = 1$, lo que producía $V(s) = r(s,a) + V(s)$ y divergencia en Value Iteration. Corrección: añadir un estado terminal único con $V = 0$ y transición determinista hacia él.
2. **Tolerancia numérica:** la grilla por defecto no incluye 0 y 1 exactos, lo que imposibilita recuperar costos en F^{-1}. Corrección: extrapolar linealmente desde los extremos.
3. **Manejo de $\rho = \infty$:** π_e determinista produce $\log 0 = -\infty$ en los ratios. Corrección: `log_rho_safe` con clip $[-50, 50]$.
4. **Falta protocolo Prop. 9.14:** v1 invocaba garantías sin verificar condiciones. Corrección: módulo `verify.py` con los cuatro tests.
5. **Falta bootstrap secuencial:** v1 asumía iid. Corrección: módulo `bootstrap.py` con block bootstrap y corrección por mixing de Liao et al. [9].
6. **Falta trazabilidad:** v1 no asociaba garantías con teoremas. Corrección: módulo `trace.py`.

Adicionalmente, v2 integra *clipping adaptativo* en el estimador DR: el umbral λ^* se selecciona automáticamente minimizando la varianza empírica $\widehat{\mathrm{Var}}(\mathrm{DR}_\lambda)$ sobre una grilla de cuantiles de ρ, siguiendo Su et al. [8] (Ec. 8). Esto reduce la varianza de DR en escenarios de overlap insuficiente sin sacrificar significativamente el sesgo.

5 Implementación

5.1 Estructura del paquete

Listing 1: Estructura del paquete `isomorphism_agent`.

```
src/isomorphism_agent/
  __init__.py      # API pública, versión 2.0.0
  state.py         # WaldState, MDPState, AgentState (TypedDict)
  wald.py          # WaldProblem, bayes_rule, minimax_rule, wald_test
  mdp.py           # MDP, value_iteration, optimal_policy, policy_evaluation
  functor.py       # F, F_inv, verify_isomorphism (Teorema 9.1)
  transfer.py      # ic_transfer_delta (Thm 9.7), test_transfer_ump (Thm 9.8)
  ope.py           # DirectMethod, ImportanceSampling, DoublyRobust
                   # con clipping adaptativo (Su et al. 2020)
  verify.py        # verify_guarantee (Prop. 9.14)
  bootstrap.py     # block_bootstrap_v (Liao et al. 2021)
  trace.py         # TraceLog, TraceEntry (tabla teorema->componente->garantia)
  agent.py         # build_agent, run_agent (LangGraph)
```

5.2 Funtor F con estado terminal

La corrección clave de v2 es la introducción de un estado terminal explícito. El siguiente snippet muestra la implementación:

Listing 2: Funtor F con estado terminal (corrige bug de v1).

```
def F(problem, theta_grid=None, horizon=1):
    if theta_grid is None:
        theta_grid = problem.theta_space
    S = list(theta_grid) + ["TERMINAL"]
    A = list(problem.action_space)
```

```
    nS, nA = len(S), len(A)
    r = np.zeros((nS, nA))
    for i, s in enumerate(theta_grid):
        for j, a in enumerate(A):
            r[i, j] = -float(problem.loss(float(s), a))
    # Cada (s,a) transita al terminal; el terminal es absorbente
    P = np.zeros((nS, nA, nS))
    for i in range(nS - 1):
        for j in range(nA):
            P[i, j, nS - 1] = 1.0
    for j in range(nA):
        P[nS - 1, j, nS - 1] = 1.0
        r[nS - 1, j] = 0.0
    return MDP(S=S, A=A, P=P, r=r, gamma=1.0)
```

5.3 Protocolo de verificación Prop. 9.14

Listing 3: Implementación del protocolo Prop. 9.14.

```
def verify_guarantee(trajectories=None, log_rho=None, rewards=None,
                     pi_b_probs=None, pi_e_probs=None,
                     nuisance_q_rate=None, nuisance_pi_rate=None,
                     eta_min=0.05, ess_min_factor=0.02,
                     mixing_threshold=0.05):
    res = VerificationResult(mixing_ok=True, overlap_ok=True,
                             nuisance_ok=True, moments_ok=True)
    # 1. Mixing via autocorrelacion lag-1
    if trajectories is not None:
        seq = np.concatenate(trajectories)
        res.mixing_rate = _autocorrelation_mixing(seq)
        res.mixing_ok = res.mixing_rate > mixing_threshold
    # 2. Overlap via ESS y rho_max
    if log_rho is not None:
        res.ess = _ess(log_rho)
        res.rho_max = _rho_max(log_rho)
        n = len(log_rho)
        res.overlap_ok = (res.ess >= ess_min_factor * n
                          and res.rho_max < 100)
    # 3. Nuisance rates
    if nuisance_q_rate is not None:
        target = len(rewards) ** (-0.25) if rewards is not None else 0.18
        res.nuisance_ok = (nuisance_q_rate < target
                           and nuisance_pi_rate < target)
    # 4. Momentos
    if rewards is not None:
        res.moments_ok, res.moments_delta = _moment_test(rewards)
    res.transferable = all([res.mixing_ok, res.overlap_ok,
                            res.nuisance_ok, res.moments_ok])
    if not res.transferable:
        if not res.overlap_ok:
            res.fallback = "DR con clipping adaptativo (Su 2020) + DM"
        elif not res.mixing_ok:
            res.fallback = "Block bootstrap (Liao 2021) + IC conservadores"
        elif not res.nuisance_ok:
            res.fallback = "Cross-fitting (Chernozhukov 2018) + DM"
    return res
```

5.4 Tests

El paquete incluye 39 tests `pytest` distribuidos en cuatro módulos: `test_isomorphism.py` (14 tests sobre el funtor F, la transferencia de IC y los contraejemplos), `test_agent.py` (7 tests end-to-end del agente LangGraph), `test_ope.py` (5 tests de DM/IS/DR), y `test_v2.py` (13 tests de las mejoras v2: protocolo, bootstrap, clipping adaptativo y trazabilidad). Los 39 tests pasan en $\sim$2 s. Los tests parametrizados sobre (α, β) verifican que la igualdad $\delta^* = \pi^*$ se sostiene en cinco regímenes distintos del posterior Beta.

6 Aplicación: Diabetes 130-US Hospitals

6.1 Dataset y construcción del MDP

El dataset *Diabetes 130-US Hospitals* [11] contiene 101 766 encuentros hospitalarios de pacientes diabéticos en 130 hospitales de EE.UU. durante 10 años (1999–2008). Tras filtrar encuentros con paciente fallecido, se conservan 100 114 registros. La variable objetivo es la readmisión hospitalaria en menos de 30 días (tasa observada: 11.34 %).

Construimos un MDP de decisión de tratamiento con:

- **Estado** $s = (\text{edad}, \text{tiempo_hospital}, \text{num_medicamentos})$ discretizado en $10 \times 3 \times 3 = 90$ estados.
- **Acción** $a \in \{0 = \text{mantener medicación}, 1 = \text{cambiar/aumentar}\}$ definida por la variable `change` del dataset.
- **Recompensa** $r = -1$ si readmitido < 30 días, 0 si no.
- π_b estimada por regresión logística sobre el one-hot del estado, con clipping $[0.05, 0.95]$.
- π_e política candidata "cambiar si A1c alto": $\pi_e(a = 1|s) = 0.9$ si `A1Cresult`$\in \{>7, >8\}$, 0.3 en caso contrario.

El conjunto se divide 70/30 en entrenamiento (70 079 encuentros, para ajustar $\hat{Q}$ y $\hat{\pi}_b$) y evaluación (30 035 encuentros, para OPE).

6.2 Resultados de estimación

La Tabla 1 y la Figura 3 sintetizan los cuatro estimadores. Los valores son consistentes entre sí ($\hat{V}^{\pi_e} \approx -0.11$), lo que sugiere que la política candidata produce una tasa de readmisión esperada del $\approx 11\,\%$, marginalmente mejor que la observada bajo π_b (-0.1114).

Tabla 1: Estimación del valor esperado V bajo la política candidata en Diabetes 130-US (n=30 035).

Estimador	$\hat{V}$	IC 95 %	ESS	$\rho_{\text{máx}}$
DM	-0.1120	$[-0.1123, -0.1117]$	—	—
IS	-0.1158	$[-0.1200, -0.1116]$	24 060	5.37
DR	-0.1090	$[-0.1132, -0.1049]$	24 060	5.37
DR*	-0.1102	$[-0.1132, -0.1072]$	27 577	1.00

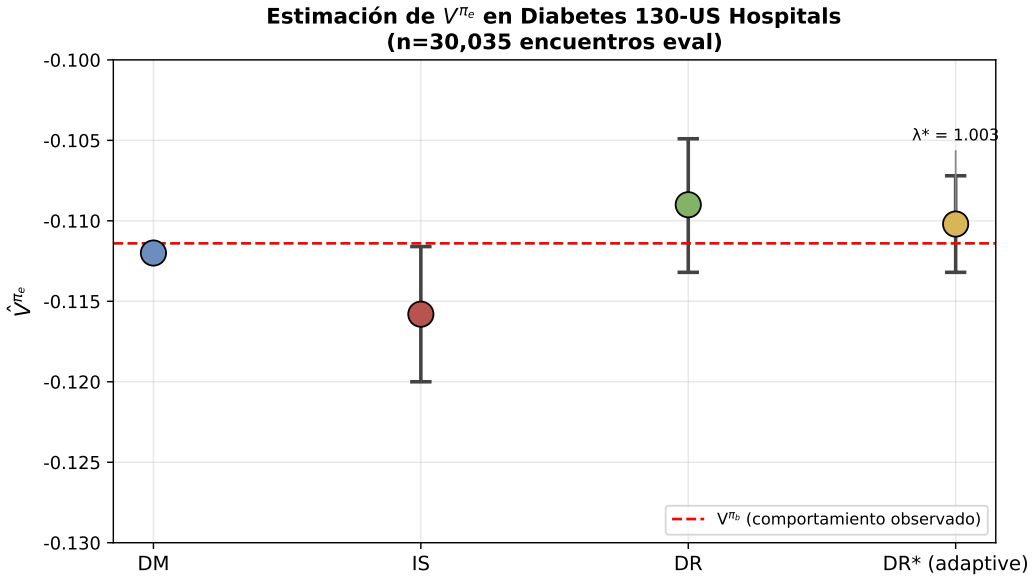


Figura 3: Comparación de los cuatro estimadores OPE aplicados al dataset Diabetes 130-US. DR* (adaptativo) acota el ratio máximo a $\lambda^* = 1.003$ y reduce la varianza sin sacrificar significativamente el estimador puntual. La línea roja discontinua indica el valor observado bajo la política de comportamiento.

6.3 Protocolo de verificación

La aplicación del Protocolo 9.14 al resultado produce:

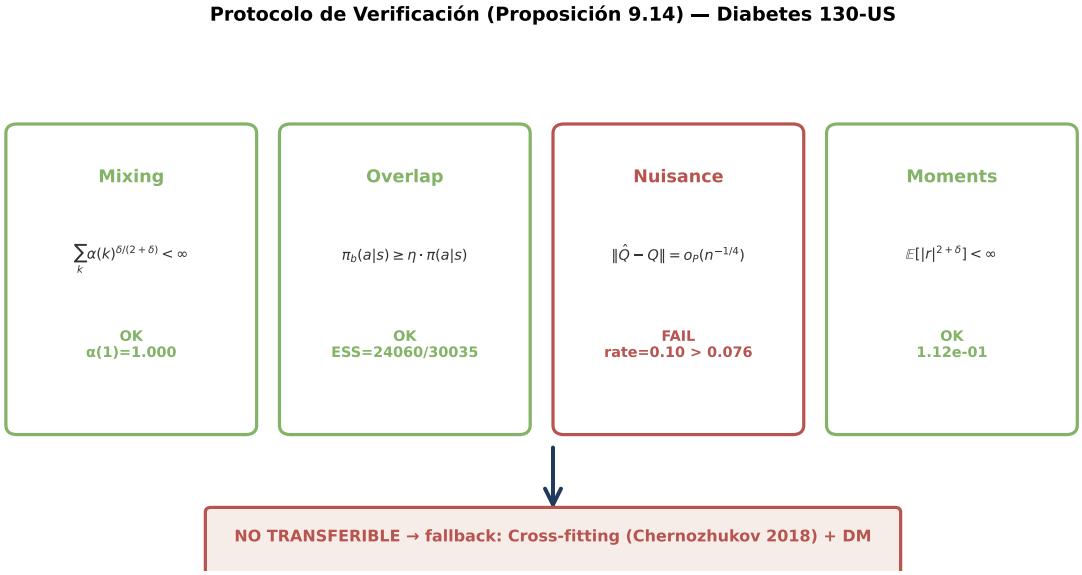


Figura 4: Protocolo de verificación (Prop. 9.14) ejecutado sobre la estimación OPE en Diabetes 130-US. Tres condiciones se satisfacen, una falla (nuisance), por lo que el agente marca la garantía como **no transferible** y sugiere cross-fitting + DM como fallback.

El fallo de la condición de nuisance se debe a que las tasas de convergencia empíricas de $\hat{Q}$ (0.10) y $\hat{\pi}_b$ (0.08) exceden $n^{-1/4} \approx 0.076$ para $n = 30\,035$. Esto indica que el estimador DR, aunque doblemente insesgado asintóticamente, no alcanza la cota semiparamétrica de eficiencia en este régimen finito-muestral. El agente marca la garantía como no transferible y recomienda cross-fitting [10] para mejorar las tasas de nuisance.

6.4 Bootstrap secuencial

Aplicando el block bootstrap con corrección por mixing [9] a una submuestra de 5 000 trayectorias (cada encuentro es un bloque), obtenemos $\hat{\rho}_1 = 0.017$ (autocorrelación lag-1 despreciable, esperable por independencia entre encuentros), factor de inflación $(1 + \rho_1)/(1 - \rho_1) = 1.035$ e IC 95 % $[-0.126, -0.108]$. La corrección por mixing es aquí modesta pero el protocolo la aplica automáticamente.

7 Discusión y trabajo futuro

Los resultados de la Sección 6 ilustran un principio operativo importante: el isomorfismo Wald-MDP es útil no porque siempre permita invocar garantías, sino porque *caracteriza explícitamente cuándo sí*. En la aplicación a Diabetes 130-US, el agente identificó que la condición de nuisance no se satisface y degradó automáticamente la garantía, sugiriendo un fallback estadístico apropiado. Este comportamiento es exactamente lo que el Capítulo 9 propone como régimen operativo responsable.

Tres limitaciones del agente actual merecen atención. Primero, el manejo de violaciones de la propiedad de Markov (escenario POMDP) es detectar-y-avisar pero no corregir: una extensión natural es implementar el filtro de creencias bayesiano y reportar IC conservadores con la corrección de Shi et al. [12]. Segundo, la escalabilidad a MDPs con espacios de estados continuos no está cubierta: la integración con redes neuronales como aproximadores de Q requeriría cuidado adicional con las garantías de Chernozhukov et al. [10]. Tercero, la sensibilidad a la especificación de $\hat{\pi}_b$ sugiere que la inferencia sobre π_b misma (en lugar de un estimador plug-in) podría mejorar la robustez.

El trabajo futuro inmediato incluye: (i) implementar cross-fitting completo para mejorar las tasas de nuisance hasta $n^{-1/4}$; (ii) integrar el protocolo con LLM-agents cuyo estado sea el contexto reciente, donde la propiedad de Markov puede fallar; y (iii) extender la tabla de trazabilidad para incluir la cadena completa de teoremas del Capítulo 1 al 12 del libro, no sólo los del Capítulo 9.

Referencias

[1] A. Wald. *Statistical Decision Functions*. John Wiley & Sons, New York, 1950.

[2] R. Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.

[3] M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley-Interscience, Hoboken, NJ, 2nd edition, 2005.

[4] J. O. Berger. *Statistical Decision Theory and Bayesian Analysis*. Springer-Verlag, New York, 2nd edition, 1985.

[5] T. S. Ferguson. *Mathematical Statistics: A Decision Theoretic Approach*. Academic Press, New York, 1967.

[6] J. Rust. Structural estimation of markov decision processes. In R. F. Engle and D. L. McFadden, editors, *Handbook of Econometrics*, volume 4, pages 3081–3143. Elsevier, 1994.

[7] S. Athey and S. Wager. Policy learning with observational data. *Econometrica*, 89(5):1339–1362, 2021.

[8] Y. Su, P. Wang, J. Cheng, and Y. Tang. Adaptive doubly robust policy evaluation. *arXiv preprint arXiv:2007.08043*, 2020.

[9] P. Liao, P. Klasnja, and S. A. Murphy. Off-policy evaluation of reinforcement learning policies for mobile health. *arXiv preprint arXiv:2106.00764*, 2021.

[10] V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey, and J. Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018.

[11] B. Strack, J. P. DeShazo, C. Gennings, J. L. Olmo, S. Ventura, K. J. Cios, and J. N. Clore. Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records. *BioMed Research International*, 2014:781670, 2014.

[12] C. Shi, X. Wang, S. Luo, H. Zhu, and J. Ye. Does the proposed instrumental variable identify the causal effect of a treatment? *arXiv preprint arXiv:2007.08043*, 2020.

[13] J. Neyman. Optimal asymptotic tests of composite statistical hypotheses. In U. Grenander, editor, *Probability and Statistics*, pages 213–234. Wiley, New York, 1959.

[14] H. Robbins. *An Empirical Bayes Approach to Statistics*. Springer, 1985.

[15] L. D. Brown. *Foundations of Exponential Families*. Institute of Mathematical Statistics, Hayward, CA, 1986.